Bird Count Analyses

Question

Assuming I have bird count data from various locations over a period of several years, how might I used Bayesian statistics and causal inference to analyse and draw conclusions from this data?

Analyzing bird count data from various locations over several years using Bayesian statistics and causal inference is a powerful approach to uncover patterns, estimate parameters, and infer causal relationships. Here's a step-by-step guide on how you might tackle this, blending Bayesian methods for uncertainty quantification with causal inference to address questions like "What factors drive bird population changes?" I'll assume your data includes bird counts, locations, time (years), and potentially covariates like temperature, habitat type, or human activity.

Step 1: Define Your Research Questions

Before diving into methods, clarify what you want to learn:

- **Descriptive**: How do bird counts vary by location and time?
- **Predictive**: Can we forecast future counts based on trends?
- **Causal**: Does a factor (e.g., temperature, habitat loss) cause changes in bird counts?

Let's assume a mix of these, with a focus on causal inference (e.g., "Does temperature affect bird counts?").

Step 2: Data Preparation

Your dataset might look like:

- `bird_count`: Number of birds observed (likely integer, possibly overdispersed).
- `location`: Categorical (e.g., site A, B, C).
- `year`: Continuous or categorical time variable.
- `covariates`: Temperature, rainfall, habitat type, etc.

Clean the data, handle missing values, and explore it:

```r
# Example in R
library(tidyverse)
```

```r
data <- read.csv("bird_data.csv")
summary(data)
ggplot(data, aes(x = year, y = bird_count, color = location)) + geom_line()
```

Step 3: Choose a Bayesian Model

Bird counts are typically non-negative integers, suggesting a Poisson or Negative Binomial likelihood (the latter if overdispersion is present). A hierarchical Bayesian model can account for location and time effects.

Model Structure

- **Likelihood**: $bird_count_{i,t} \sim Poisson(\lambda_{i,t})$, where i is location and t is year.
- **Log-link for rate**: $\log(\lambda_{i,t}) = \beta_0 + \beta_1 \cdot \text{temperature}_{i,t} + \alpha_i + \gamma_t$
  - β_0: Overall intercept.
  - β_1: Effect of temperature.
  - α_i: Location-specific random effect, $\alpha_i \sim N(0, \sigma_\alpha^2)$.
  - γ_t: Year-specific random effect, $\gamma_t \sim N(0, \sigma_\gamma^2)$.
- **Priors**:
  - $\beta_0, \beta_1 \sim N(0, 10)$ (weakly informative).
  - $\sigma_\alpha, \sigma_\gamma \sim \text{Cauchy}(0, 5)$ (positive, light-tailed).

Implementation in R with Stan

Using the `rstanarm` package for simplicity:

```r
library(rstanarm)
model <- stan_glmer(bird_count ~ temperature + (1 | location) + (1 | year),
                    family = poisson(link = "log"),
                    data = data,
                    prior = normal(0, 10),
                    prior_intercept = normal(0, 10))
summary(model)
```

This fits a Poisson regression with random effects, estimating posterior distributions for β_1 (temperature effect) and variance parameters.

Step 4: Bayesian Inference

- **Posterior Distributions**: Extract means, credible intervals (e.g., 95%) for parameters.

```r
posterior <- as.data.frame(model)
mean_beta1 <- mean(posterior$temperature)
ci_beta1 <- quantile(posterior$temperature, c(0.025, 0.975))
cat("Temperature effect:", mean_beta1, "95% CI:", ci_beta1)
```

- **Interpretation**: If the credible interval for β_1 excludes 0, temperature likely influences bird counts.
- **Predictions**: Use `posterior_predict()` to simulate bird counts under different temperatures.

Step 5: Causal Inference

Bayesian modeling alone estimates associations, not causation. To infer causality (e.g., "Temperature causes bird count changes"), combine it with causal inference techniques.

Approach 1: Directed Acyclic Graphs (DAGs)

- Draw a DAG to hypothesize causal relationships:
  - Temperature → Bird Count
  - Habitat → Bird Count
  - Temperature → Habitat (confounding?)
- Use `dagitty` in R to identify adjustment sets:

```r
library(dagitty)
dag <- dagitty("dag { Temperature -> BirdCount; Habitat -> BirdCount; Temperature -> Habitat }")
adjustmentSets(dag, "Temperature", "BirdCount")  # Might suggest adjusting for Habitat
```

Approach 2: Propensity Score or Covariate Adjustment

- Adjust for confounders (e.g., habitat) in the Bayesian model:

```r
model_causal <- stan_glmer(bird_count ~ temperature + habitat + (1 | location) + (1 | year),
                           family = poisson(link = "log"),
                           data = data)
```

- Check if β_1 changes significantly—indicating confounding.

Approach 3: Difference-in-Differences (DiD)

- If you have an intervention (e.g., a conservation policy at some locations), compare treated vs. control locations over time:

```r
model_did <- stan_glmer(bird_count ~ temperature + treated * post + (1 | location) + (1 | year),
                        family = poisson(link = "log"),
                        data = data)
```

  - `treated`: 1 if location received intervention, 0 otherwise.
  - `post`: 1 if after intervention year, 0 before.
  - Interaction term estimates causal effect.

Step 6: Validate and Interpret

- **Posterior Predictive Checks**: Ensure the model fits the data:

```r
pp_check(model)  # Compare observed vs. predicted counts
```

- **Sensitivity Analysis**: Test different priors or likelihoods (e.g., Negative Binomial).
- **Causal Conclusions**: If β_1's credible interval excludes 0 after adjustment, you might infer "Temperature causally affects bird counts," with uncertainty quantified.

Step 7: Draw Conclusions

- **Descriptive**: "Bird counts increased by X% at location A from 2018–2023."
- **Predictive**: "Given a 2°C rise, we predict a Y% decline in counts (95% CI: Z–W%)."
- **Causal**: "A 1°C increase reduces bird counts by K% (95% CI: M–N%), likely causal after adjusting for habitat."

Tools and Extensions

- **Packages**: `rstanarm`, `brms` (more flexible), `tidybayes` (posterior visualization).
- **Extensions**: Add spatial effects (e.g., `spBayes`), time trends (e.g., AR(1) terms), or interaction terms for complex hypotheses.

This approach leverages Bayesian flexibility for uncertainty and causal inference to move beyond correlation. Let me know if you'd like to refine this for your specific data!

Question

One difficulty with the data is that although often there are precise counts, sometimes I am given ranges (eg 200-250), and other times it is simply marked as the species being 'present'. It isn't clear to me how I should use the items marked as 'present' in an analysis.

Dealing with heterogeneous data like precise counts, ranges (e.g., 200–250), and vague "present" observations is a common challenge in ecological datasets like your bird counts. This mix of data types complicates a straightforward Poisson or Negative Binomial model, but Bayesian statistics offers flexible ways to handle it by treating these as different observation processes or latent states. Here's how you can approach this, including how to incorporate "present" observations meaningfully.

Step 1: Understand the Data Types

- **Precise Counts**: Exact integers (e.g., 75 birds).
- **Ranges**: Interval-censored data (e.g., 200–250 birds).
- **"Present"**: Indicates at least 1 bird, but no upper bound or precise count (right-censored or presence-only data).

The "present" case is tricky because it provides minimal quantitative information—it confirms the species exists at the location and time but doesn't specify abundance. You'll need to model this as a lower bound or a different observation type.

Step 2: Model Framework

A Bayesian hierarchical model can unify these data types by:

1. Modeling the **true (latent) count** for each observation as a Poisson or Negative Binomial process.
2. Defining **observation processes** that reflect how the data were recorded (precise, range, or "present").

Latent Count Model

Assume the true bird count $N_{i,t}$ at location i and time t follows:
$N_{i,t} \sim \text{Poisson}(\lambda_{i,t})$
$\log(\lambda_{i,t}) = \beta_0 + \beta_1 \cdot \text{temperature}_{i,t} + \alpha_i + \gamma_t$
(As before, with random effects for location and year.)

Observation Process

- **Precise Counts**: The observed count $Y_{i,t} = N_{i,t}$.
- **Ranges**: The observed range $[L_{i,t}, U_{i,t}]$ constrains $N_{i,t}$ such that $L_{i,t} \leq N_{i,t} \leq U_{i,t}$.

- "Present": The observed data implies $N_{i,t} \geq 1$, but no upper limit.

Step 3: Handling "Present" Observations

The "present" data can be incorporated in a few ways, depending on your goals:

Option 1: Treat as Censored Data (Minimal Count = 1)

- Model "present" as $N_{i,t} \geq 1$, meaning the true count is at least 1 but unknown beyond that.
- In Bayesian terms, this is left-truncated or right-censored data (depending on perspective). You integrate over all possible values $N_{i,t} \geq 1$ consistent with the Poisson rate $\lambda_{i,t}$.
- Likelihood Contribution: The probability $P(N_{i,t} \geq 1) = 1 - P(N_{i,t} = 0) = 1 - e^{-\lambda_{i,t}}$.

Option 2: Impute a Small Count (e.g., 1)

- Assign $Y_{i,t} = 1$ for "present" cases, treating it as a minimum observed count. This assumes "present" implies a low abundance, which may or may not hold biologically.
- Downside: This could bias results downward if "present" sometimes means larger counts.

Option 3: Presence-Absence Model

- Treat "present" as a binary outcome (1 = present, 0 = absent if you have absence data) and fit a separate logistic regression:
  $\text{logit}(P(\text{present}_{i,t})) = \beta_0 + \beta_1 \cdot \text{temperature}_{i,t} + \alpha_i$
- Combine this with a count model for precise/range data, linking the two via shared parameters or a joint process (e.g., $\lambda_{i,t}$ influences both presence and abundance).

Recommended Approach: Option 1 (Censored Data)

- It's the most flexible and least assumptive. It uses the information "present" provides (at least 1 bird) without forcing an arbitrary count or splitting the analysis.

Step 4: Implementation in R with Stan

Using `rstan` (more flexible than `rstanarm` for custom likelihoods), here's a simplified model:

Stan Code

```
data {
  int<lower=0> N_precise;  // Number of precise counts
  int<lower=0> N_range;    // Number of range observations
```

```
  int<lower=0> N_present;  // Number of "present" observations
  int<lower=0> Y_precise[N_precise];  // Precise count data
  int<lower=0> L_range[N_range];      // Lower bounds for ranges
  int<lower=0> U_range[N_range];      // Upper bounds for ranges
  vector[N_precise + N_range + N_present] temp;  // Temperature covariate
  int<lower=0> N_loc;  // Number of locations
  int<lower=1> loc[N_precise + N_range + N_present];  // Location index
}
parameters {
  real beta0;              // Intercept
  real beta1;              // Temperature effect
  vector[N_loc] alpha;     // Location effects
  real<lower=0> sigma_alpha;  // SD of location effects
  int<lower=0> N_latent[N_range + N_present];  // Latent counts for ranges and present
}
model {
  // Priors
  beta0 ~ normal(0, 10);
  beta1 ~ normal(0, 10);
  sigma_alpha ~ cauchy(0, 5);
  alpha ~ normal(0, sigma_alpha);

  // Latent rates
  vector[N_precise + N_range + N_present] log_lambda;
  for (i in 1:(N_precise + N_range + N_present)) {
    log_lambda[i] = beta0 + beta1 * temp[i] + alpha[loc[i]];
  }

  // Likelihoods
  Y_precise ~ poisson_log(log_lambda[1:N_precise]);  // Precise counts
  for (i in 1:N_range) {  // Range data
    target += poisson_log_lpmf(N_latent[i] | log_lambda[N_precise + i]);
    N_latent[i] ~ uniform(L_range[i], U_range[i]);  // Constrain within range
  }
  for (i in 1:N_present) {  // "Present" data
    target += poisson_log_lccdf(0 | log_lambda[N_precise + N_range + i]);  // P(N >= 1)
  }
}
```

R Code to Run

```
library(rstan)
# Example data
data <- list(
  N_precise = 3, N_range = 2, N_present = 2,
  Y_precise = c(75, 100, 50),
  L_range = c(200, 150), U_range = c(250, 175),
  temp = c(20, 22, 19, 21, 23, 18, 20),
  N_loc = 2, loc = c(1, 1, 2, 2, 1, 2, 1)
)
```

```r
fit <- stan(file = "bird_model.stan", data = data)
print(fit, pars = c("beta0", "beta1", "sigma_alpha"))
```

Step 5: Interpretation

- **Precise Counts**: Directly inform $\lambda_{i,t}$.
- **Ranges**: The model imputes $N_{i,t}$ within bounds, updating $\lambda_{i,t}$.
- **"Present"**: Contributes via $P(N_{i,t} \geq 1)$, weakly informing the rate (less weight than precise counts).
- **Causal Inference**: Adjust for confounders (e.g., habitat) as before, using the estimated β_1.

Step 6: Practical Considerations

- **Overdispersion**: If counts vary more than a Poisson allows, switch to Negative Binomial (replace `poisson_log` with `neg_binomial_2_log` in Stan).
- **Missing Data**: If "present" reflects observer effort, add an effort covariate.
- **Validation**: Use posterior predictive checks to ensure the model captures all data types.

Conclusion

By treating "present" as censored data ($N_{i,t} \geq 1$), you include it without over-assuming abundance, letting the Bayesian framework balance its influence with precise and range data. This approach retains all observations, quantifies uncertainty, and supports causal inference when paired with a DAG or adjustment. Let me know if you'd like to tweak this further!

> **Question**
>
> You mention observer effort, and this is definitely a factor, given that we are not dealing with regularly scheduled surveys, but simply counts from whenever amateur birdwatchers happen to be in the area and record the information on the platform. How might we account for this?

Accounting for observer effort in your bird count data is crucial, especially since your dataset comes from opportunistic observations by amateur birdwatchers rather than systematic surveys. Variable effort (e.g., time spent, area covered, or observer skill) can bias counts—higher effort might yield more birds simply because more time or space was sampled, not because of true abundance. Bayesian statistics offers a flexible way to

model this by incorporating effort as an offset, a covariate, or a latent process. Here's how you can approach it, given your irregular, citizen-science context.

Step 1: Conceptualize Observer Effort

Effort could represent:

- **Time**: Hours spent watching.
- **Area**: Size of the surveyed region.
- **Observer Activity**: Number of birdwatchers or visits.
- **Skill**: Observer experience (harder to quantify).

Since your data comes from a platform, explicit effort metrics (e.g., hours) might not be recorded. If absent, you'll need to infer or approximate it from proxies like the number of species reported, observation frequency, or metadata (e.g., date, time of day).

Step 2: Modeling Approaches

Here are three ways to account for effort, depending on available data:

Option 1: Effort as an Offset

- **When to Use**: You have a direct measure of effort (e.g., hours spent) or a proxy (e.g., number of submissions per location/day).
- **How**: Include effort as an offset in the Poisson model, adjusting the expected count rate per unit of effort.
- **Model**:
  $N_{i,t} \sim \text{Poisson}(\lambda_{i,t} \cdot E_{i,t})$
  $\log(\lambda_{i,t}) = \beta_0 + \beta_1 \cdot \text{temperature}_{i,t} + \alpha_i + \gamma_t$
  - $E_{i,t}$: Effort (e.g., hours), logged in the model as $\log(E_{i,t})$.
  - $\lambda_{i,t}$: Birds per unit effort.
- **Stan Code Addition**:

```
data {
  vector[N_precise + N_range + N_present] log_effort;  // Log of effort
}
model {
  vector[N_precise + N_range + N_present] log_lambda;
  for (i in 1:(N_precise + N_range + N_present)) {
    log_lambda[i] = beta0 + beta1 * temp[i] + alpha[loc[i]] + log_effort[i];
  }
  Y_precise ~ poisson_log(log_lambda[1:N_precise]);
  // ... (rest as before)
```

```
}
```

- **Pros**: Simple, interpretable (rate per effort).
- **Cons**: Requires effort data, which you might not have.

Option 2: Effort as a Covariate

- **When to Use**: Effort is partially observed or proxied (e.g., number of visits to a location in a year).
- **How**: Add effort as a predictor with its own coefficient:
  $\log(\lambda_{i,t}) = \beta_0 + \beta_1 \cdot \text{temperature}_{i,t} + \beta_2 \cdot \text{effort}_{i,t} + \alpha_i + \gamma_t$
- **Stan Code Addition**:

```
parameters {
  real beta2;  // Effort coefficient
}
model {
  beta2 ~ normal(0, 10);  // Prior
  for (i in 1:(N_precise + N_range + N_present)) {
    log_lambda[i] = beta0 + beta1 * temp[i] + beta2 * effort[i] + alpha[loc[i]];
  }
}
```

- **Pros**: Allows effort's effect to be estimated, not assumed.
- **Cons**: Still needs a proxy; β_2 might soak up unrelated variation if effort is noisy.

Option 3: Latent Effort (Hierarchical Model)

- **When to Use**: No direct effort data, but you suspect it varies by location, time, or observer behavior.
- **How**: Treat effort as a latent variable inferred from the data, possibly with a prior distribution:
  $N_{i,t} \sim \text{Poisson}(\lambda_{i,t} \cdot E_{i,t})$
  $\log(E_{i,t}) \sim N(\mu_E, \sigma_E^2)$
  $\log(\lambda_{i,t}) = \beta_0 + \beta_1 \cdot \text{temperature}_{i,t} + \alpha_i + \gamma_t$
- **Stan Code**:

```
parameters {
  vector[N_precise + N_range + N_present] log_effort;  // Latent log-effort
  real mu_E;            // Mean log-effort
  real<lower=0> sigma_E;  // SD of log-effort
}
model {
```

```
  mu_E ~ normal(0, 1);       // Prior on mean effort
  sigma_E ~ cauchy(0, 2);    // Prior on effort variation
  log_effort ~ normal(mu_E, sigma_E);  // Random effect for effort
  for (i in 1:(N_precise + N_range + N_present)) {
    log_lambda[i] = beta0 + beta1 * temp[i] + alpha[loc[i]] + log_effort[i];
  }
  Y_precise ~ poisson_log(log_lambda[1:N_precise]);
  // ... (range and present as before)
}
```

- **Pros**: Handles missing effort flexibly; infers it from patterns in counts.
- **Cons**: Computationally intensive; relies on model assumptions (e.g., effort's distribution).

Step 3: Proxies for Effort

Since effort isn't explicitly recorded, construct proxies:

- **Visit Frequency**: Count observations per location-year (e.g., 5 reports in 2020 at site A = higher effort).

```r
library(dplyr)
effort_proxy <- data %>%
  group_by(location, year) %>%
  summarise(effort = n(), .groups = "drop")
data <- left_join(data, effort_proxy, by = c("location", "year"))
```

- **Species Richness**: Number of species reported per observation (more species might indicate more effort).
- **Time of Day/Season**: Weight observations by peak bird activity periods (e.g., morning, migration season) if timestamps exist.

Step 4: Choose an Approach

- **If You Have Proxies**: Use Option 1 (offset) or 2 (covariate). Offset is preferred if the proxy scales linearly with expected counts (e.g., visits ≈ effort).
- **If No Proxies**: Use Option 3 (latent effort). It's speculative but leverages the hierarchical structure to estimate variation.

For opportunistic data, **Option 2 with visit frequency** is often practical:

```r
```

```r
# Add effort as covariate in rstanarm
library(rstanarm)
model <- stan_glmer(bird_count ~ temperature + effort + (1 | location) + (1 | year),
                    family = poisson(link = "log"),
                    data = data)
summary(model)
```

Step 5: Causal Inference Adjustment

Effort might confound your causal question (e.g., temperature → effort → bird counts). Include it in your DAG:

- Temperature → Bird Count
- Temperature → Effort → Bird Count
  Adjust for effort in the model to isolate temperature's direct effect.

Step 6: Validate

- **Posterior Checks**: Do higher-effort observations predict higher counts? If not, the proxy or model may need tweaking.
- **Sensitivity**: Test different effort proxies or prior assumptions.

Conclusion

Given your citizen-science context, I'd start with **visit frequency as a covariate** (Option 2). It's simple, uses available data, and corrects for observation bias without overcomplicating the model. If that's insufficient (e.g., counts don't scale with visits), switch to a latent effort approach. Either way, effort adjustment strengthens your causal claims by reducing confounding from irregular sampling. Let me know if you need help coding a specific proxy!

> **Question**
>
> Please provide an example of the predictive prior using the bird count data, with code in R.

To illustrate the **prior predictive distribution** using your bird count data, I'll set up a Bayesian model based on the context we've discussed (e.g., Poisson likelihood for counts, with covariates like temperature and random effects for location and year). The prior predictive distribution will show the range of bird counts we might expect *before seeing the data*, based on our prior beliefs about the parameters and the model structure.

Since your data includes precise counts, ranges, and "present" observations with variable observer effort, I'll simplify slightly for clarity, focusing on a Poisson model with temperature and effort, and then generate prior predictive samples in R. You can adapt this to include ranges or "present" data later.

Step 1: Define the Model

Let's assume:

- **Likelihood**: $N_{i,t} \sim \text{Poisson}(\lambda_{i,t})$, where $N_{i,t}$ is the bird count at location i and year t.
- **Log-linear model**:
  $\log(\lambda_{i,t}) = \beta_0 + \beta_1 \cdot \text{temperature}_{i,t} + \beta_2 \cdot \text{effort}_{i,t} + \alpha_i$
  - β_0: Intercept.
  - β_1: Temperature effect.
  - β_2: Effort effect (log-scale, assuming effort is a covariate like visit frequency).
  - α_i: Location random effect, $\alpha_i \sim N(0, \sigma_\alpha^2)$.
- **Priors**:
  - $\beta_0 \sim N(2, 1)$ (baseline log-rate, expecting 5–20 birds per unit effort).
  - $\beta_1 \sim N(0, 0.5)$ (temperature effect, centered at 0, moderate uncertainty).
  - $\beta_2 \sim N(1, 0.5)$ (effort scales counts, centered near 1 for log-linearity).
  - $\sigma_\alpha \sim \text{Half-Cauchy}(0, 1)$ (location variability).

The **prior predictive distribution** is:
$p(N_{i,t}) = \int p(N_{i,t}|\lambda_{i,t})\, p(\lambda_{i,t}|\beta_0, \beta_1, \beta_2, \alpha_i)\, p(\beta_0, \beta_1, \beta_2, \alpha_i, \sigma_\alpha)\, d\beta_0\, d\beta_1\, d\beta_2\, d\alpha_i\, d\sigma_\alpha$
In practice, we simulate this by sampling from the priors and generating counts.

Step 2: R Code for Prior Predictive Distribution

Here's how to simulate it in R:

```r
# Load necessary libraries
library(tidyverse)

# Set seed for reproducibility
set.seed(123)

# Simulate data structure (fake covariates for illustration)
n_obs <- 100  # Number of observations
n_locations <- 5  # Number of unique locations
data <- data.frame(
  location = sample(1:n_locations, n_obs, replace = TRUE),
  temperature = rnorm(n_obs, mean = 20, sd = 5),  # Temp in Celsius
  effort = rexp(n_obs, rate = 0.5)  # Effort (e.g., visits, exponential for skew)
```

```r
)

# Function to generate prior predictive samples
prior_predictive <- function(n_samples, data) {
  # Sample parameters from priors
  beta0 <- rnorm(n_samples, mean = 2, sd = 1)  # Intercept
  beta1 <- rnorm(n_samples, mean = 0, sd = 0.5)  # Temp effect
  beta2 <- rnorm(n_samples, mean = 1, sd = 0.5)  # Effort effect
  sigma_alpha <- abs(rcauchy(n_samples, scale = 1))  # Location variability

  # Sample location effects for each unique location
  alpha <- matrix(nrow = n_samples, ncol = n_locations)
  for (j in 1:n_samples) {
    alpha[j, ] <- rnorm(n_locations, mean = 0, sd = sigma_alpha[j])
  }

  # Generate prior predictive counts
  counts <- matrix(nrow = n_samples, ncol = nrow(data))
  for (i in 1:n_samples) {
    log_lambda <- beta0[i] +
                  beta1[i] * data$temperature +
                  beta2[i] * log(data$effort) +  # Log effort for positivity
                  alpha[i, data$location]
    lambda <- exp(log_lambda)
    counts[i, ] <- rpois(nrow(data), lambda)
  }

  return(counts)
}

# Generate 1000 prior predictive samples
n_samples <- 1000
prior_counts <- prior_predictive(n_samples, data)

# Summarize results
prior_summary <- data.frame(
  observation = rep(1:nrow(data), each = n_samples),
  count = as.vector(prior_counts)
) %>%
  group_by(observation) %>%
  summarise(
    mean_count = mean(count),
    lower_95 = quantile(count, 0.025),
    upper_95 = quantile(count, 0.975)
  )

# Add original covariates for context
prior_summary <- bind_cols(prior_summary, data)

# Plot prior predictive distribution
ggplot(prior_summary, aes(x = observation, y = mean_count)) +
  geom_point() +
  geom_errorbar(aes(ymin = lower_95, ymax = upper_95), width = 0.2) +
  labs(title = "Prior Predictive Distribution of Bird Counts",
       y = "Predicted Count (95% Interval)", x = "Observation Index") +
```

```
  theme_minimal()

# Histogram of all counts
ggplot(data.frame(count = as.vector(prior_counts)), aes(x = count)) +
  geom_histogram(binwidth = 5, fill = "skyblue", color = "black") +
  labs(title = "Histogram of Prior Predictive Counts", x = "Bird Count")
+
```

Step 3: Explanation of the Code

1. **Fake Data**: I created a small dataset with temperature, effort, and location to mimic your structure. Replace this with your real data.
2. **Prior Sampling**:
   - Sample $\beta_0, \beta_1, \beta_2$ from Normal priors.
   - Sample σ_α from a Half-Cauchy (positive only).
   - Sample location effects α_i for each location.
3. **Count Generation**:
   - Compute $\log(\lambda_{i,t})$ for each observation and sample.
   - Exponentiate to get $\lambda_{i,t}$.
   - Draw counts from $\text{Poisson}(\lambda_{i,t})$.
4. **Visualization**:
   - Plot mean counts with 95% intervals per observation.
   - Show a histogram of all simulated counts.

Step 4: Interpretation

- **Range of Counts**: The histogram shows the spread of possible bird counts (e.g., 0 to hundreds), reflecting prior uncertainty.
- **Plausibility**: Check if these counts align with your intuition:
  - If counts seem too high (e.g., thousands), tighten priors (e.g., lower β_0).
  - If too low, adjust priors upward or increase σ_α.
- **Effort Effect**: Higher effort increases λ, so counts scale with $\log(\text{effort})$.

For your data with ranges and "present":

- **Ranges**: Constrain simulated counts (e.g., reject samples outside 200–250).
- **"Present"**: Filter to $N_{i,t} \geq 1$ or use $1 - \text{ppois}(0, \lambda)$ for probability.

Step 5: Refine for Your Data

Replace `data` with your real dataset (e.g., with `bird_count`, `temperature`, `effort`, `location`). Adjust priors based on domain knowledge (e.g., typical bird counts per

visit). This prior predictive check helps ensure your model's assumptions are reasonable before fitting.

Let me know if you'd like to tweak priors or add complexity (e.g., year effects)!

> **Question**
>
> Given that some winters are harsher than others, and recently there has been incidences of bird flu affecting bird counts, how might I add year effects to this model?

To account for year-specific effects like harsh winters or bird flu outbreaks impacting bird counts, you can extend the Bayesian model by adding a **year random effect** (or fixed effect, depending on your goals) to the prior predictive framework. This captures annual variation beyond what temperature and effort explain, such as extreme weather or disease. I'll modify the previous model and R code to include year effects, assuming your data spans multiple years.

Step 1: Update the Model

We'll add a year-specific random effect γ_t to the log-rate:

- **Likelihood**: $N_{i,t} \sim \text{Poisson}(\lambda_{i,t})$.
- **Updated Log-linear Model**:
  $\log(\lambda_{i,t}) = \beta_0 + \beta_1 \cdot \text{temperature}_{i,t} + \beta_2 \cdot \text{effort}_{i,t} + \alpha_i + \gamma_t$
  - $\alpha_i \sim N(0, \sigma_\alpha^2)$: Location random effect (as before).
  - $\gamma_t \sim N(0, \sigma_\gamma^2)$: Year random effect (new).
  - σ_γ: Standard deviation of year effects, reflecting how much counts vary annually.
- **Priors**:
  - $\beta_0, \beta_1, \beta_2$: Same as before.
  - $\sigma_\alpha, \sigma_\gamma \sim \text{Half-Cauchy}(0, 1)$ (both positive, moderate variability).

The **prior predictive distribution** now integrates over γ_t as well, allowing year-to-year fluctuations (e.g., a harsh winter or bird flu in 2023 reduces $\lambda_{i,t}$ for that year).

Step 2: Updated R Code

Here's the modified code with year effects:

```r
# Load necessary libraries
library(tidyverse)

```

```r
# Set seed for reproducibility
set.seed(123)

# Simulate data structure with years
n_obs <- 100  # Number of observations
n_locations <- 5  # Number of unique locations
n_years <- 6  # Number of years (e.g., 2018-2023)
data <- data.frame(
  location = sample(1:n_locations, n_obs, replace = TRUE),
  year = sample(2018:2023, n_obs, replace = TRUE),  # Years as integers
  temperature = rnorm(n_obs, mean = 20, sd = 5),   # Temp in Celsius
  effort = rexp(n_obs, rate = 0.5)                 # Effort (e.g., visits)
)

# Function to generate prior predictive samples with year effects
prior_predictive <- function(n_samples, data) {
  # Sample parameters from priors
  beta0 <- rnorm(n_samples, mean = 2, sd = 1)    # Intercept
  beta1 <- rnorm(n_samples, mean = 0, sd = 0.5)  # Temp effect
  beta2 <- rnorm(n_samples, mean = 1, sd = 0.5)  # Effort effect
  sigma_alpha <- abs(rcauchy(n_samples, scale = 1))  # Location variability
  sigma_gamma <- abs(rcauchy(n_samples, scale = 1))  # Year variability

  # Unique years in data
  unique_years <- sort(unique(data$year))
  n_years <- length(unique_years)

  # Sample location and year effects
  alpha <- matrix(nrow = n_samples, ncol = n_locations)
  gamma <- matrix(nrow = n_samples, ncol = n_years)
  for (j in 1:n_samples) {
    alpha[j, ] <- rnorm(n_locations, mean = 0, sd = sigma_alpha[j])
    gamma[j, ] <- rnorm(n_years, mean = 0, sd = sigma_gamma[j])
  }

  # Map year indices to data rows
  year_idx <- match(data$year, unique_years)

  # Generate prior predictive counts
  counts <- matrix(nrow = n_samples, ncol = nrow(data))
  for (i in 1:n_samples) {
    log_lambda <- beta0[i] +
                  beta1[i] * data$temperature +
                  beta2[i] * log(data$effort) +
                  alpha[i, data$location] +
                  gamma[i, year_idx]  # Add year effect
    lambda <- exp(log_lambda)
    counts[i, ] <- rpois(nrow(data), lambda)
  }

  return(counts)
}

# Generate 1000 prior predictive samples
n_samples <- 1000
```

```r
prior_counts <- prior_predictive(n_samples, data)

# Summarize results
prior_summary <- data.frame(
  observation = rep(1:nrow(data), each = n_samples),
  count = as.vector(prior_counts),
  year = rep(data$year, each = n_samples),
  location = rep(data$location, each = n_samples)
) %>%
  group_by(observation, year, location) %>%
  summarise(
    mean_count = mean(count),
    lower_95 = quantile(count, 0.025),
    upper_95 = quantile(count, 0.975),
    .groups = "drop"
)

# Add original covariates for context
prior_summary <- bind_cols(prior_summary, select(data, temperature, effort))

# Plot prior predictive by year
ggplot(prior_summary, aes(x = year, y = mean_count, color = factor(location))) +
  geom_point() +
  geom_errorbar(aes(ymin = lower_95, ymax = upper_95), width = 0.2) +
  labs(title = "Prior Predictive Bird Counts by Year and Location",
       y = "Predicted Count (95% Interval)", x = "Year") +
  theme_minimal()

# Histogram of all counts
ggplot(data.frame(count = as.vector(prior_counts)), aes(x = count)) +
  geom_histogram(binwidth = 5, fill = "skyblue", color = "black") +
  labs(title = "Histogram of Prior Predictive Counts with Year Effects", x = "Bird Count") +
```

Step 3: Explanation of Changes

1. **Data Structure**:
   - Added `year` as a variable (e.g., 2018–2023). Replace with your actual years.
2. **Year Effects**:
   - Sample σ_γ from a Half-Cauchy prior.
   - Generate γ_t for each unique year, adding it to $\log(\lambda_{i,t})$.
   - Map each observation's year to its corresponding γ_t using `match()`.
3. **Prior Predictive**:
   - Counts now reflect year-to-year variation. A year with a negative γ_t (e.g., harsh winter or bird flu) lowers counts, while a positive γ_t raises them.
4. **Visualization**:
   - Plot shows counts by year and location, with 95% intervals capturing prior uncertainty, including year effects.

Step 4: Interpretation

- **Year Variation**: The γ_t term introduces annual fluctuations. If σ_γ is large, counts vary more across years (e.g., a bird flu year might drop counts significantly).
- **Plausibility Check**:
  - Are the simulated counts reasonable for harsh winters (lower) or disease years?
  - Adjust σ_γ's prior (e.g., scale = 0.5) if variation seems too extreme.
- **Real Data**: With your actual years, γ_t will later be estimated to reflect observed annual patterns (e.g., 2022 bird flu).

Step 5: Alternative: Fixed Year Effects

If you suspect specific years (e.g., known bird flu outbreaks) have unique impacts, use fixed effects:

- Replace $\gamma_t \sim N(0, \sigma_\gamma^2)$ with $\gamma_t \sim N(0, 10)$ for each year, treating them as independent parameters.
- In the code, remove σ_γ and sample γ directly:

```r
gamma[j, ] <- rnorm(n_years, mean = 0, sd = 10)  # Wider prior for fixed effects
```

Step 6: Next Steps

- **Real Data**: Use your bird count data (precise, ranges, "present") and actual years.
- **Posterior**: After fitting, γ_t's posterior will show which years deviate (e.g., negative γ_{2023} for bird flu).
- **Covariates**: Add winter severity or flu incidence as covariates if data exists, reducing reliance on random effects.

This setup lets the prior predictive reflect annual shocks like harsh winters or bird flu, preparing you to fit and interpret these effects later. Let me know if you want to refine further!